



L7B: Streams I; Programming Language Processing

CS1101S: Programming Methodology

Sanka Rasnayaka

2nd October, 2026



Course Overview

- ▶ Unit 1 — Functions (textbook Chapter 1)
 - Getting acquainted with the elements of programming, using **functional abstraction**
 - Learning to read programs, and using the **substitution model**
 - Example applications: runes, curves



Course Overview

- ▶ Unit 1 — Functions (textbook Chapter 1)
 - Getting acquainted with the elements of programming, using **functional abstraction**
 - Learning to read programs, and using the **substitution model**
 - Example applications: runes, curves
- ▶ Unit 2 — Data (textbook Chapter 2)
 - Getting familiar with data and **data abstraction**: pairs, llists, trees, **streams**
 - Searching in llists and trees, processing of llists and trees
 - Example applications: sound processing



Course Overview

- ▶ Unit 1 — Functions (textbook Chapter 1)
 - Getting acquainted with the elements of programming, using **functional abstraction**
 - Learning to read programs, and using the **substitution model**
 - Example applications: runes, curves
- ▶ Unit 2 — Data (textbook Chapter 2)
 - Getting familiar with data and **data abstraction**: pairs, llists, trees, **streams**
 - Searching in llists and trees, processing of llists and trees
 - Example applications: sound processing
- ▶ Unit 3 — State



Course Overview

- ▶ Unit 1 — Functions (textbook Chapter 1)
 - Getting acquainted with the elements of programming, using **functional abstraction**
 - Learning to read programs, and using the **substitution model**
 - Example applications: runes, curves
- ▶ Unit 2 — Data (textbook Chapter 2)
 - Getting familiar with data and **data abstraction**: pairs, llists, trees, **streams**
 - Searching in llists and trees, processing of llists and trees
 - Example applications: sound processing
- ▶ Unit 3 — State
- ▶ Unit 4 — Beyond
 - Streams II ...



Outline

- ▶ Streams
- ▶ Programming Language Processing



The Basics (1)

- ▶ Can we implement conditionals using functions?



The Basics (1)

- ▶ Can we implement conditionals using functions?
- ▶ What about implementing `E2 if E1 else E3` as



The Basics (1)

- ▶ Can we implement conditionals using functions?
- ▶ What about implementing `E2 if E1 else E3` as
- ▶ `cond(E1, E2, E3)`

The Basics (1)

- ▶ Can we implement conditionals using functions?
- ▶ What about implementing `E2 if E1 else E3` as
- ▶ `cond(E1, E2, E3)`
- ▶ `where`

The Basics (1)

- ▶ Can we implement conditionals using functions?
- ▶ What about implementing `E2 if E1 else E3` as
- ▶ `cond(E1, E2, E3)`
- ▶ where

```
def cond(x, y, z):  
    if x:  
        return y  
    else:  
        return z
```



The Basics (2)

- ▶ How about implementing `E2 if E1 else E3` as

The Basics (2)

- ▶ How about implementing `E2 if E1 else E3` as
- ▶ `cond(E1, lambda: E2, lambda: E3)`



The Basics (2)

- ▶ How about implementing `E2 if E1 else E3` as
- ▶ `cond(E1, lambda: E2, lambda: E3)`
- ▶ `where`



The Basics (2)

- ▶ How about implementing `E2 if E1 else E3` as
- ▶ `cond(E1, lambda: E2, lambda: E3)`
- ▶ where

```
def cond(x, y, z):  
    if x:  
        return y()  
    else:  
        return z()
```





Delayed Evaluation

```
cond(E1, lambda: E2, lambda: E3)
```

- ▶ Main idea:
 - We delayed the evaluation of `E2` and `E3` until we had enough information to decide which one was needed



Delayed Evaluation

```
cond(E1, lambda: E2, lambda: E3)
```

- ▶ Main idea:
 - We delayed the evaluation of `E2` and `E3` until we had enough information to decide which one was needed
- ▶ Instrument of delay:
 - Functions allow us to describe an activity without actually doing the activity



Continuations are Delayed Computations

- ▶ Deferred operations are “stored” in a function.



Continuations are Delayed Computations

- ▶ Deferred operations are “stored” in a function.
- ▶ When base case is reached, the stored deferred operations get applied.



Continuations are Delayed Computations

- ▶ Deferred operations are “stored” in a function.
- ▶ When base case is reached, the stored deferred operations get applied.

```
def fact_helper(n, cont):  
    return (cont(1) if n == 1  
            else fact_helper(  
                n - 1,  
                lambda x: cont(n * x)))  
def factorial(n):  
    return fact_helper(n, lambda x: x)
```





A Simple Example

```
def f(x):  
    return lambda: x + 1
```





A Simple Example

```
def f(x):  
    return lambda: x + 1
```

```
# returns a function that stores computation  
y = f(99)
```



A Simple Example

```
def f(x):  
    return lambda: x + 1
```

```
# returns a function that stores computation  
y = f(99)  
  
# two weeks later  
z = y()
```



A Simple Example

```
def f(x):  
    return lambda: x + 1
```

```
# returns a function that stores computation  
y = f(99)  
  
# two weeks later  
z = y()
```




A Simple Example

```
def f(x):  
    return lambda: x + 1
```

```
# returns a function that stores computation  
y = f(99)  
  
# two weeks later  
z = y()
```

- Functions are like pickles



A Simple Example

```
def f(x):  
    return lambda: x + 1
```

```
# returns a function that stores computation  
y = f(99)  
  
# two weeks later  
z = y()
```

- ▶ Functions are like pickles
- ▶ Open the jar (apply the function) only when you need the value



Sum of Primes

```
# returns the sum of all prime  
# numbers in the range [a, b].  
def sum_primes(a, b):  
    def iter(count, accum):  
        if count > b:  
            return accum  
        elif is_prime(count):  
            return iter(count + 1, count + accum)  
        else:  
            return iter(count + 1, accum)  
    return iter(a, 0)
```



Sum of Primes, with list processing

```
# returns the sum of all prime  
# numbers in the range [a, b].  
def sum_primes(a, b):  
    return reduce(  
        lambda x, y: x + y,  
        0,  
        filter(is_prime, enum_llist(a, b))  
    )
```



Extreme example: Second prime number after 10000

```
head(tail(filter(  
    is_prime,  
    enum_llist(10000, 1000000)  
)))
```



Extreme example: Second prime number after 10000

```
head(tail(filter(  
    is_prime,  
    enum_llist(10000, 1000000)  
)))
```

- ▶ What is wrong here?
- ▶ We only want the second prime number between 10000 and 1M



Idea of Streams

- ▶ Delayed sequences
 - Our pairs contain a **data item** as **head** (as usual), but a **function** as **tail** that can be activated when needed



Idea of Streams

- ▶ Delayed sequences
 - Our pairs contain a **data item** as **head** (as usual), but a **function** as **tail** that can be activated when needed
- ▶ Streams
 - A **stream** is either the empty llist, or a pair whose tail is a nullary function that returns a stream



Idea of Streams

- ▶ Delayed sequences
 - Our pairs contain a **data item** as **head** (as usual), but a **function** as **tail** that can be activated when needed
- ▶ Streams
 - A **stream** is either the empty llist, or a pair whose tail is a nullary function that returns a stream
- ▶ Streams: typed version
 - A stream of some type is either the empty llist, or a pair whose head is of that type and whose tail is a nullary function that returns a stream of that type



Idea of Streams

- ▶ Delayed sequences
 - Our pairs contain a **data item** as **head** (as usual), but a **function** as **tail** that can be activated when needed
- ▶ Streams
 - A **stream** is either the empty llist, or a pair whose tail is a nullary function that returns a stream
- ▶ Streams: typed version
 - A stream of some type is either the empty llist, or a pair whose head is of that type and whose tail is a nullary function that returns a stream of that type
- ▶ Stream discipline
 - Like llist discipline, now using streams



Example Streams

```
# An empty stream  
s1 = None
```



Example Streams

```
# An empty stream  
s1 = None  
  
# A stream with element 1  
s2 = pair(1, lambda: None)
```



Example Streams

```
# An empty stream
s1 = None

# A stream with element 1
s2 = pair(1, lambda: None)

# A stream with elements 1, 2, 3
s3 = pair(1,
          lambda: pair(2,
                        lambda: pair(3,
                                      lambda: None))))
```



An “Infinite” Stream

```
def ones_stream():  
    return pair(1, ones_stream)  
  
ones = ones_stream()
```





An “Infinite” Stream

```
def ones_stream():  
    return pair(1, ones_stream)  
  
ones = ones_stream()
```



`head(ones) ⇒ 1`



An “Infinite” Stream

```
def ones_stream():  
    return pair(1, ones_stream)  
  
ones = ones_stream()
```



`head(ones) ⇒ 1`

`head(tail(ones)()) ⇒ 1`



An “Infinite” Stream

```
def ones_stream():  
    return pair(1, ones_stream)  
  
ones = ones_stream()
```



`head(ones) ⇒ 1`

`head(tail(ones)()) ⇒ 1`

`head(tail(tail(ones)())()) ⇒ 1`



A Convenient Function: `stream_tail`

```
def stream_tail(stream):  
    return tail(stream())
```





A Convenient Function: `stream_tail`

```
def stream_tail(stream):  
    return tail(stream)()
```



```
ones = ones_stream()
```



A Convenient Function: `stream_tail`

```
def stream_tail(stream):  
    return tail(stream)()
```



```
ones = ones_stream()
```

```
head(ones)  $\Rightarrow$  1
```



A Convenient Function: `stream_tail`

```
def stream_tail(stream):  
    return tail(stream)()
```



```
ones = ones_stream()
```

```
head(ones)  $\Rightarrow$  1
```

```
head(stream_tail(ones))  $\Rightarrow$  1
```



A Convenient Function: `stream_tail`

```
def stream_tail(stream):  
    return tail(stream)()
```



```
ones = ones_stream()
```

```
head(ones)  $\Rightarrow$  1
```

```
head(stream_tail(ones))  $\Rightarrow$  1
```

```
head(stream_tail(stream_tail(ones)))  $\Rightarrow$  1
```



Streams Are Lazy Llists

```
def enum_stream(low, hi):  
    if low > hi:  
        return None  
    else:  
        return pair(low,  
                     lambda: enum_stream(low + 1, hi))  
  
s = enum_stream(1, 100)
```

`head(s) ⇒ 1`



Streams Are Lazy Llists

```
def enum_stream(low, hi):  
    if low > hi:  
        return None  
    else:  
        return pair(low,  
                    lambda: enum_stream(low + 1, hi))  
  
s = enum_stream(1, 100)
```

`head(s) ⇒ 1`

`head(stream_tail(s)) ⇒ 2`



Streams Are Lazy Llists

```
def enum_stream(low, hi):  
    if low > hi:  
        return None  
    else:  
        return pair(low,  
                    lambda: enum_stream(low + 1, hi))  
  
s = enum_stream(1, 100)
```

`head(s) ⇒ 1`

`head(stream_tail(s)) ⇒ 2`

`head(stream_tail(stream_tail(s))) ⇒ 3`



A Useful Function: `stream_ref`

```
def stream_ref(s, n):  
    if n == 0:  
        return head(s)  
    else:  
        return stream_ref(stream_tail(s), n - 1)  
  
s = enum_stream(1, 100)
```





A Useful Function: stream_ref

```
def stream_ref(s, n):  
    if n == 0:  
        return head(s)  
    else:  
        return stream_ref(stream_tail(s), n - 1)  
  
s = enum_stream(1, 100)
```

`stream_ref(s, 0) ⇒ 1`



A Useful Function: stream_ref

```
def stream_ref(s, n):  
    if n == 0:  
        return head(s)  
    else:  
        return stream_ref(stream_tail(s), n - 1)  
  
s = enum_stream(1, 100)
```

`stream_ref(s, 0) ⇒ 1`

`stream_ref(s, 10) ⇒ 11`



A Useful Function: stream_ref

```
def stream_ref(s, n):  
    if n == 0:  
        return head(s)  
    else:  
        return stream_ref(stream_tail(s), n - 1)  
  
s = enum_stream(1, 100)
```

`stream_ref(s, 0) ⇒ 1`

`stream_ref(s, 10) ⇒ 11`

`stream_ref(s, 99) ⇒ 100`



More Useful Functions: stream_map

```
def stream_map(f, s):  
    if is_none(s):  
        return None  
    else:  
        return pair(  
            f(head(s)),  
            lambda: stream_map(f, stream_tail(s)))
```



More Useful Functions: stream_filter

```
def stream_filter(p, s):  
    if is_none(s):  
        return None  
    elif p(head(s)):  
        return pair(  
            head(s),  
            lambda: stream_filter(  
                p, stream_tail(s)))  
    else:  
        return stream_filter(p, stream_tail(s))
```



Back to the extreme example

```
result = head(stream_tail(stream_filter(  
    is_prime,  
    enum_stream(10000, 1000000))))
```




Back to the extreme example

```
result = head(stream_tail(stream_filter(  
    is_prime,  
    enum_stream(10000, 1000000))))
```

- ▶ General idea
 - Only compute what is needed
 - Be lazy!



More Examples: integers_from

```
def integers_from(n):  
    return pair(n, lambda: integers_from(n + 1))  
  
integers = integers_from(1)
```

`stream_ref(integers, 0)  $\Rightarrow$  1`



More Examples: integers_from

```
def integers_from(n):  
    return pair(n, lambda: integers_from(n + 1))  
  
integers = integers_from(1)
```



`stream_ref(integers, 0) ⇒ 1`

`stream_ref(integers, 10) ⇒ 11`



More Examples: integers_from

```
def integers_from(n):  
    return pair(n, lambda: integers_from(n + 1))  
  
integers = integers_from(1)
```



`stream_ref(integers, 0) ⇒ 1`

`stream_ref(integers, 10) ⇒ 11`

`stream_ref(integers, 99) ⇒ 100`



More Examples: no_fours

```
def is_divisible(x, y):  
    return x % y == 0  
  
no_fours = stream_filter(  
    lambda x: not is_divisible(x, 4),  
    integers)
```



More Examples: no_fours

```
def is_divisible(x, y):  
    return x % y == 0  
  
no_fours = stream_filter(  
    lambda x: not is_divisible(x, 4),  
    integers)
```

`stream_ref(no_fours, 3) ⇒ 5`

More Examples: no_fours

```
def is_divisible(x, y):  
    return x % y == 0  
  
no_fours = stream_filter(  
    lambda x: not is_divisible(x, 4),  
    integers)
```

`stream_ref(no_fours, 3) ⇒ 5`

`stream_ref(no_fours, 100) ⇒ 134`



From Streams to Llists



From Streams to Llists

```
def eval_stream(s, n):  
    if n == 0:  
        return None  
    else:  
        return pair(head(s),  
                     eval_stream(stream_tail(s), n - 1))
```



From Streams to Llists

```
def eval_stream(s, n):  
    if n == 0:  
        return None  
    else:  
        return pair(head(s),  
                     eval_stream(stream_tail(s), n - 1))
```

```
eval_stream(no_fours, 10)  
#[1, [2, [3, [5, [6, [7, [9, [10, [11, [13, None]]]]]]]]]
```



Stream Processing

- ▶ Like llists, except
 - Lazy (on-demand)
 - Wrap tail in a function
 - Use `stream_tail` instead of `tail`



Stream Processing

- ▶ Like llists, except
 - Lazy (on-demand)
 - Wrap tail in a function
 - Use `stream_tail` instead of `tail`
- ▶ Stream support in Python §3 (wait for unit 3!)
 - STREAMS provides pre-declared functions for stream processing
 - Examples: `stream_tail`, `stream_map`, `stream_filter`



Check-In time!

Look for Check-In I6A in the Source Academy!



Outline

- ▶ Streams
- ▶ Programming Language Processing
 - **T-Diagrams**
 - Interpreters
 - Compilers
 - Combinations
 - Programming the LEGO Bricks



T-Diagrams (Tombstone Diagrams)



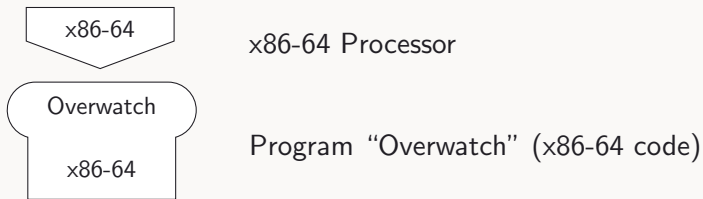
T-Diagrams (Tombstone Diagrams)

x86-64

x86-64 Processor

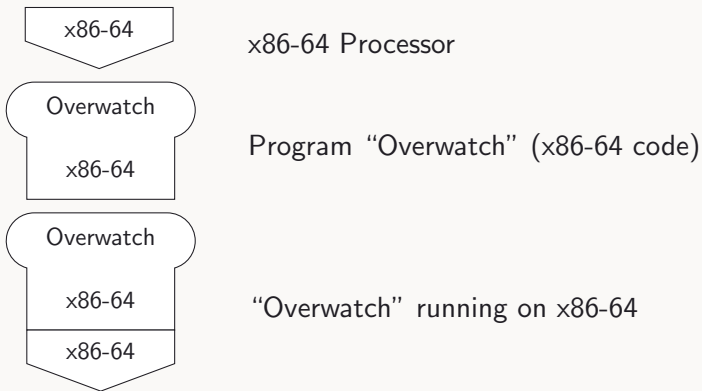


T-Diagrams (Tombstone Diagrams)





T-Diagrams (Tombstone Diagrams)





Running an app on iPhone 18



Running an app on iPhone 18

A20 Pro

A20 Pro, the processor of iPhone 18



Running an app on iPhone 18

A20 Pro

A20 Pro, the processor of iPhone 18

Tik Tok

Program “Tik Tok” (A20 build)

A20 Pro



Running an app on iPhone 18

A20 Pro

A20 Pro, the processor of iPhone 18

Tik Tok

Program “Tik Tok” (A20 build)

A20 Pro

Tik Tok

“Tik Tok” running on A20 Pro

A20 Pro

A20 Pro



Outline

- ▶ Streams
- ▶ Programming Language Processing
 - T-Diagrams
 - **Interpreters**
 - Compilers
 - Combinations
 - Programming the LEGO Bricks



Interpreter

- Interpreter is a program that executes another program



Interpreter

- ▶ Interpreter is a program that executes another program
- ▶ The interpreter's **implementation language** is the language in which the interpreter is written



Interpreter

- ▶ Interpreter is a program that executes another program
- ▶ The interpreter's **implementation language** is the language in which the interpreter is written
- ▶ The interpreter's **interpreted language** is the language in which the programs are written which the interpreter can execute



Interpreter

- ▶ Interpreter is a program that executes another program
- ▶ The interpreter's **implementation language** is the language in which the interpreter is written
- ▶ The interpreter's **interpreted language** is the language in which the programs are written which the interpreter can execute
- ▶ Teaser for Unit 4

The most fundamental idea in programming

The evaluator (interpreter), which determines the meaning of statements in a language, is just another program.



Interpreters

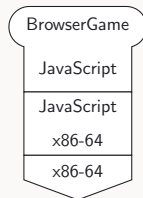
JavaScript

x86-64

Chrome browser for PC, seen as an interpreter for JavaScript,
written in x86-64 machine code



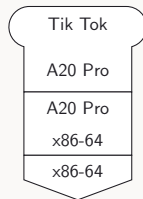
Running JavaScript on Chrome



The browser acts as an interpreter for JavaScript.



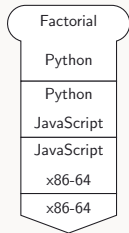
Hardware Emulation



“Tik Tok” app running on a PC using hardware emulator



Running Python §2 in Source Academy using Stepper



Source Academy stepper: layer between your programs and Chrome's native JavaScript



Outline

- ▶ Streams
- ▶ Programming Language Processing
 - T-Diagrams
 - Interpreters
 - **Compilers**
 - Combinations
 - Programming the LEGO Bricks



Compilers

► Definition:



Compilers

- ▶ Definition:
- ▶ A compiler is a program that translates from one language (the from-language) to another language (the to-language).



Compilers

- ▶ Definition:
- ▶ A compiler is a program that translates from one language (the from-language) to another language (the to-language).



Compilers

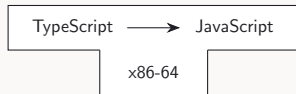
- ▶ Definition:
- ▶ A compiler is a program that translates from one language (the from-language) to another language (the to-language).

The second most fundamental idea in programming

A compiler, which translates programs from one language to another, is just another program.



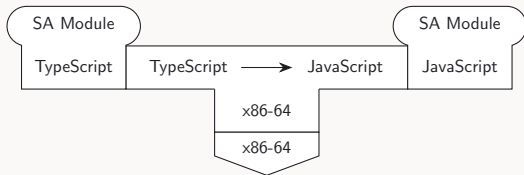
T-Diagram of Compiler



TypeScript-to-JavaScript compiler written in x86-64 machine code



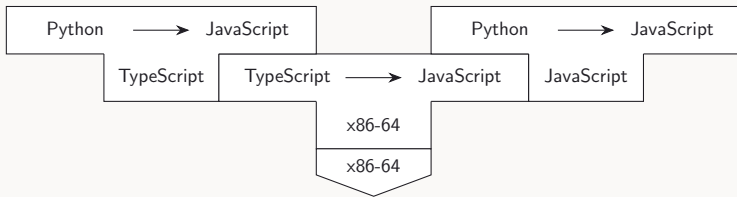
Compiling a program (Modules in Source Academy)



Compiling a "SA Module" from TypeScript to JavaScript



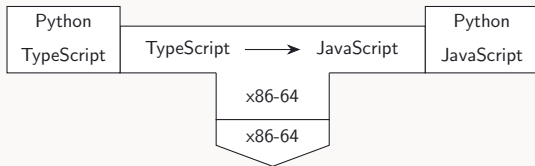
Compiling our Source Compiler



Compiling Python-to-JavaScript compiler from TypeScript to JavaScript



Compiling the Stepper



Compiling stepper tool from TypeScript to JavaScript



Outline

- ▶ Streams
- ▶ Programming Language Processing
 - T-Diagrams
 - Interpreters
 - Compilers
 - **Combinations**
 - Programming the LEGO Bricks



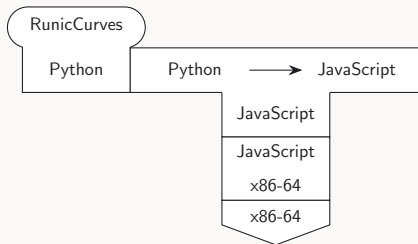
A typical Source Academy session

RunicCurves

Python



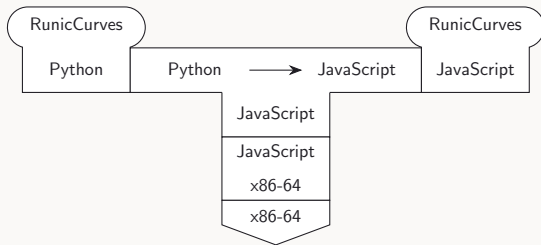
A typical Source Academy session



Compiling “RunicCurves” from Python to JavaScript in browser, and then running the JavaScript program in browser.



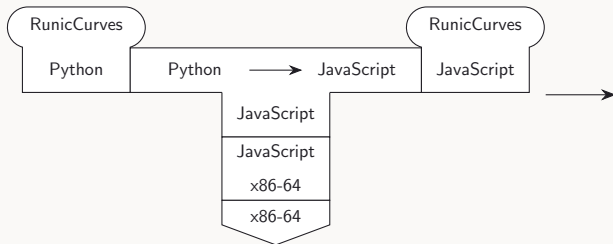
A typical Source Academy session



Compiling “RunicCurves” from Python to JavaScript in browser, and then running the JavaScript program in browser.



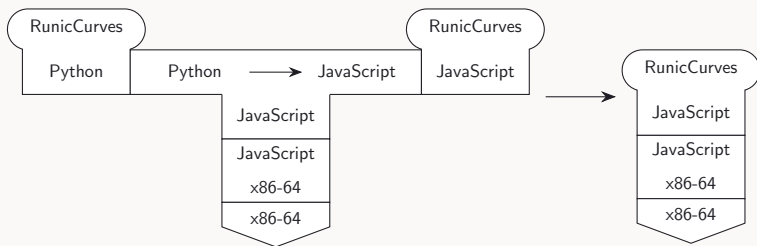
A typical Source Academy session



Compiling “RunicCurves” from Python to JavaScript in browser, and then running the JavaScript program in browser.



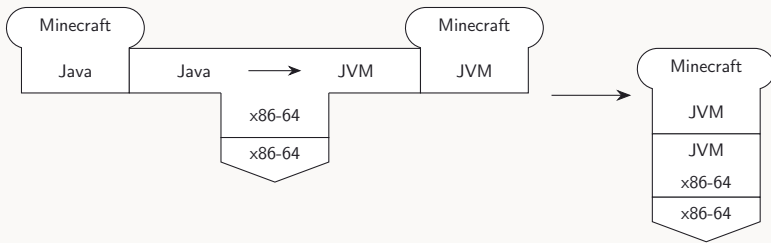
A typical Source Academy session



Compiling “RunicCurves” from Python to JavaScript in browser, and then running the JavaScript program in browser.



Typical Execution of Java Programs



Compiling “Minecraft” from Java to JVM, and then running the JVM program in JVM interpreter.



Recall: Symbolic Evaluation

- ▶ We represented functions / expressions with data structures



Recall: Symbolic Evaluation

- ▶ We represented functions / expressions with data structures
- ▶ Example expression:



Recall: Symbolic Evaluation

- ▶ We represented functions / expressions with data structures
- ▶ Example expression:

```
my_exp = make_sum(make_product("x", "x"),  
                  make_sum("x", 4))
```

Recall: Symbolic Evaluation

- ▶ We represented functions / expressions with data structures
- ▶ Example expression:

```
my_exp = make_sum(make_product("x", "x"),  
                  make_sum("x", 4))
```

- ▶ Symbolic evaluation: `eval_symbolic(my_exp, "x", 3)`



Recall: Symbolic Evaluation

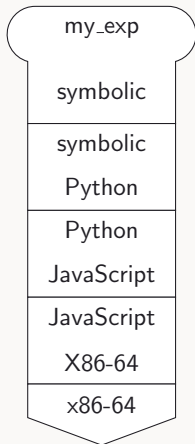
- ▶ We represented functions / expressions with data structures
- ▶ Example expression:

```
my_exp = make_sum(make_product("x", "x"),  
                  make_sum("x", 4))
```

- ▶ Symbolic evaluation: `eval_symbolic(my_exp, "x", 3)`
- ▶ should return 16



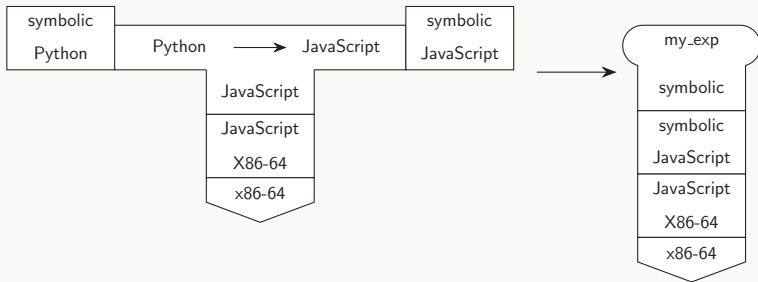
eval_symbolic on Stepper



Evaluating `my_exp` using the `eval_symbolic` interpreter, which is interpreted by the Stepper, which is interpreted by the web browser.



eval_symbolic on SA



Running `eval_symbolic` on SA by compiling

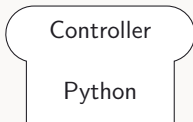


Outline

- ▶ Streams
- ▶ Programming Language Processing
 - T-Diagrams
 - Interpreters
 - Compilers
 - Combinations
 - **Programming the LEGO Bricks**



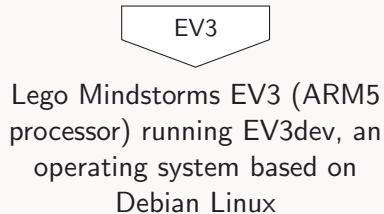
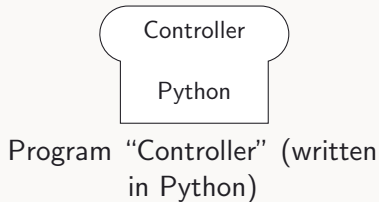
Programming Lego Mindstorms with Python



Program “Controller” (written
in Python)

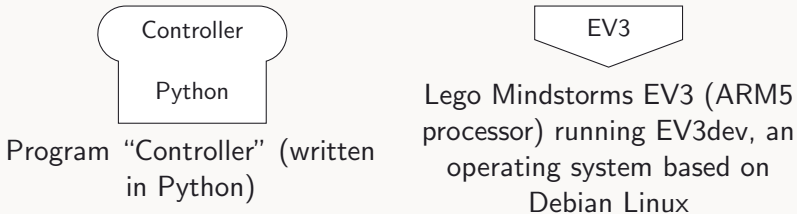


Programming Lego Mindstorms with Python





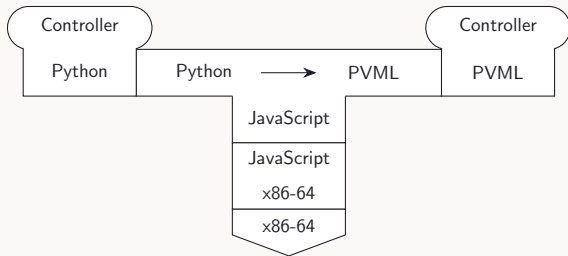
Programming Lego Mindstorms with Python



- How to run Python on EV3?

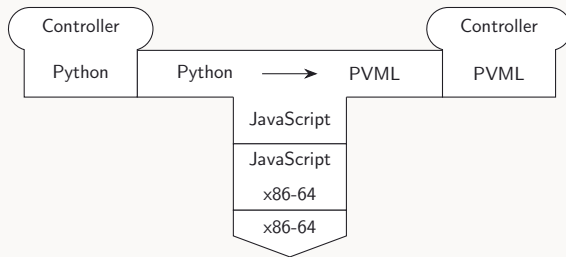


PVML to the rescue!





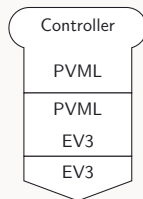
PVML to the rescue!



Compiling “controller” from Python to PVML



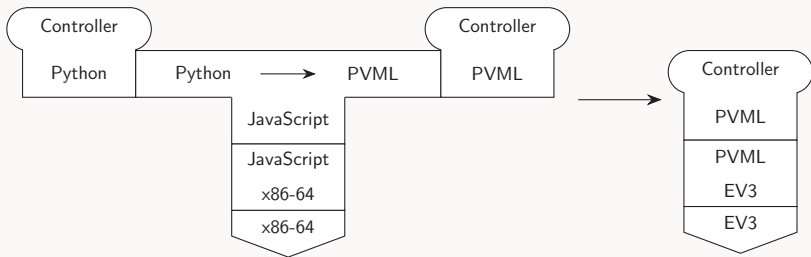
PVML on EV3



Running PVML program on EV3 brick



Overview: Python for Robotics



Compiling “controller” from Python to PVML on a PC, and then running the PVML program on the EV3 brick.



Summary

- ▶ Streams delay their tails, computing values only when needed
- ▶ Laziness supports potentially infinite data and avoids unnecessary work
- ▶ T-diagrams describe programs, machines, interpreters, and compilers
- ▶ Real systems commonly combine interpretation and compilation